
Aprendizado de Máquina

— Aprendizado supervisionado e não supervisionado;
Principais algoritmos e aplicações —



Artificial Intelligence

Machine Learning

Deep Learning

Rule Based Systems

Game Playing

Knowledge Representation and Reasoning

Support Vector
Machines

Linear Regression

Logistic Regression

MLP

GAN

Autoencoders

CNN

RBFN

RNN

K-Means Clustering

Gaussian Process
Regression

Random Forest

Cognitive Modeling

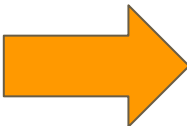
Planning

Search Algorithms

Propositional Calculus



Mudança de Paradigma - Regras para Dados

- Programar a solução -> aprender a solução a partir de exemplos
 - Permite a IA resolver problemas de altíssima complexidade
 - Reconhecimento de Imagem
 - Tradução de linguagem
 - Diagnóstico médico
- 
- Impossíveis de resolver com regras codificadas manualmente

Aplicações do dia a dia

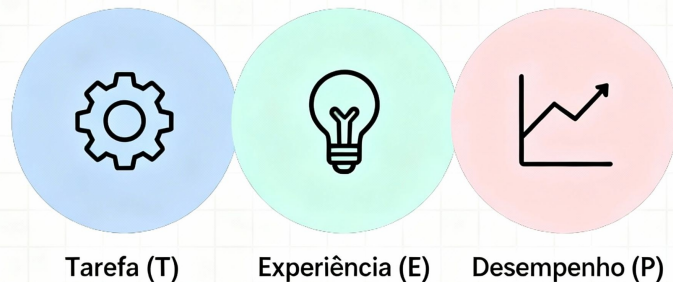


- **Netflix e Spotify** → recomendação personalizada
- **Gmail** → filtro de SPAM
- **Medicina** → diagnóstico automático por imagens

Machine Learning tradicional

- Algoritmos aprendem com dados históricos.
 - Usar o que já aconteceu para estimar o que vai acontecer
 - Técnicas em que o modelo aprende padrões a partir de dados passados para fazer previsões ou classificações.
- Algoritmos que aprendem com dados históricos para prever ou classificar informações futuras.

Definição (Mitchell, 1997)



*Um programa de computador aprende com a experiência **E**, em relação a uma tarefa **T** e uma medida de desempenho **P**, se seu desempenho em **T**, medido por **P**, melhora com a experiência **E**.*

Exemplo:

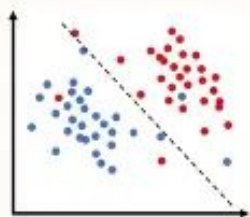
- **Tarefa (T)**: classificar e-mails SPAM/não SPAM
- **Experiência (E)**: base de e-mails rotulados
- **Desempenho (P)**: taxa de acerto

Tipos de Aprendizado (Russell & Norvig, 2013)

Tipo	O que aprende	Exemplos Reais
Supervisionado	Com rótulos	Prever preço de casa, diagnóstico médico
Não supervisionado	Sem rótulos	Agrupar clientes por perfil, análise de clusters, PCA
Reforço	Tentativa e erro	Treinar robô, AlphaGo, controles em videogames

TIPOS DE APRENDIZAGEM DE MÁQUINA

Supervisionado



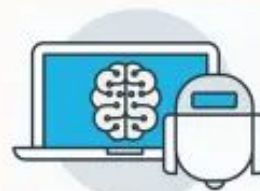
Dados rotulados

Não supervisionado



Dados não rotulados

Auto supervisionado



Modelo rotula os dados

Por reforço

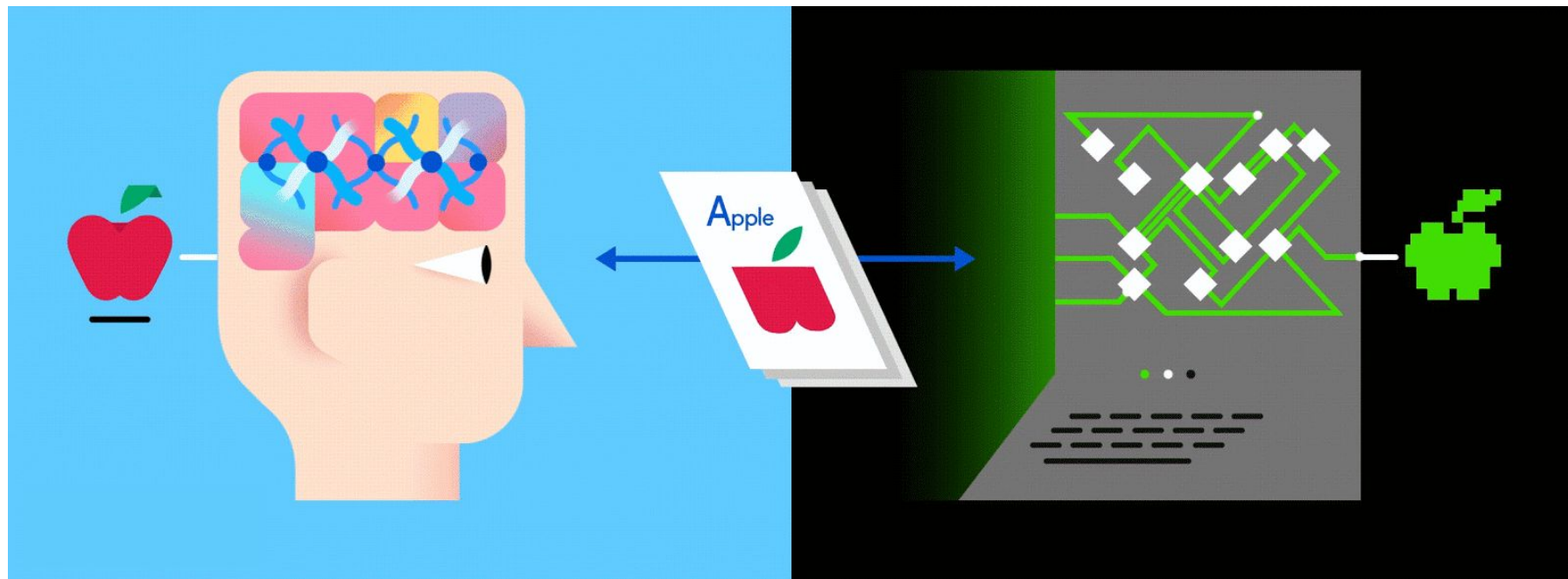


Aprende com os erros

Aprendizado Supervisionado

- Aprende a partir de um conjunto de dados de treinamento que é rotulado
 - cada exemplo de entrada já conhece a resposta correta, ou o "rótulo" (label);
- Subtipos principais:
 - **Classificação:** Prever um rótulo de uma categoria discreta
 - Qual a classe deste item?
 - Ex.: este email é *spam* ou *não spam*? Esta transação é *fraudulenta* ou *legítima*?
 - **Regressão:** Prever um valor numérico contínuo
 - Qual o valor deste item?
 - Ex: Qual será o *preço* desta casa? Qual será a *temperatura* amanhã?

Aprendizado Supervisionado

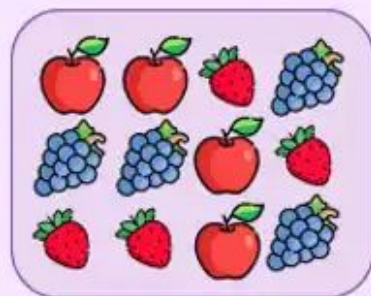


Fonte: <https://techcrunch.com/2017/04/13/neural-networks-made-easy/>

Aprendizado supervisionado

🍇 Experiência adquirida:

Dados de entrada



Rótulos



Predição



Fonte: Google images, 2022

- Predição: tolerância à estresses abióticos e bióticos;
- Produtividade;
- Potencial: futuras cultivares.

(MOHRI *et al.*, 2018; PARMLEY *et al.*, 2019; RAD & RAFEZI, 2020; JO, 2021)

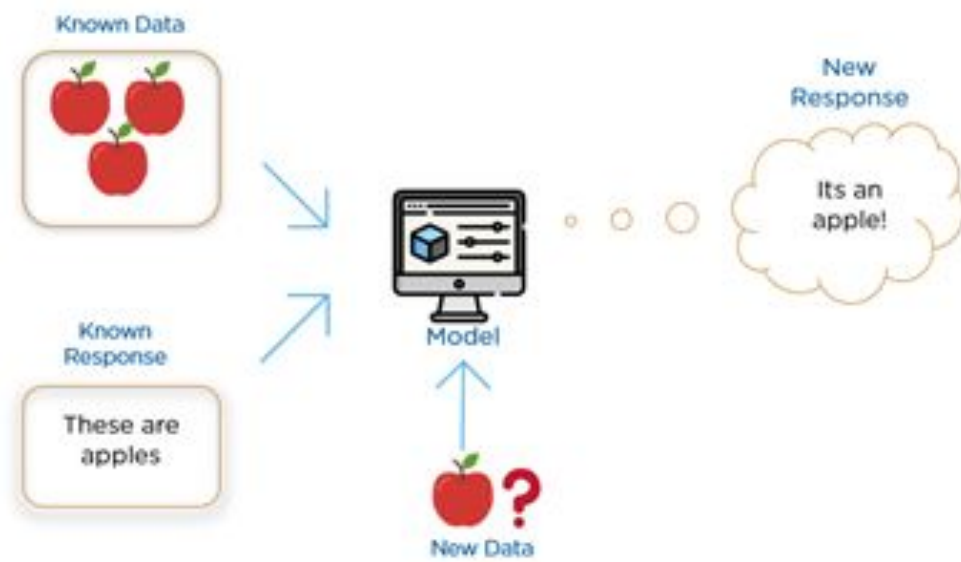
Aprendizado Supervisionado

Exemplos:

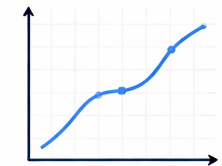
- Previsão de preços de casas (regressão)
- Classificação de flores Íris (classificação)

Principais algoritmos:

- Regressão Linear
- Árvores de Decisão
- k-NN
- SVM

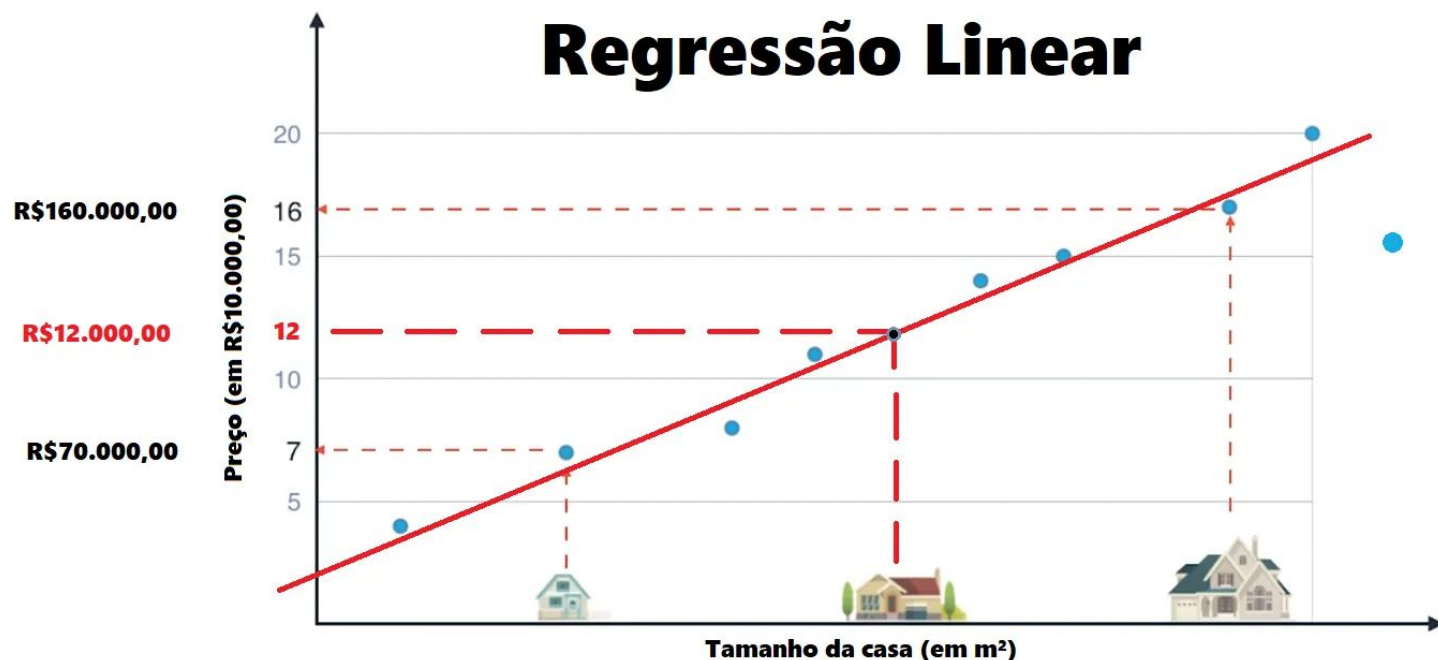


Regressão Linear



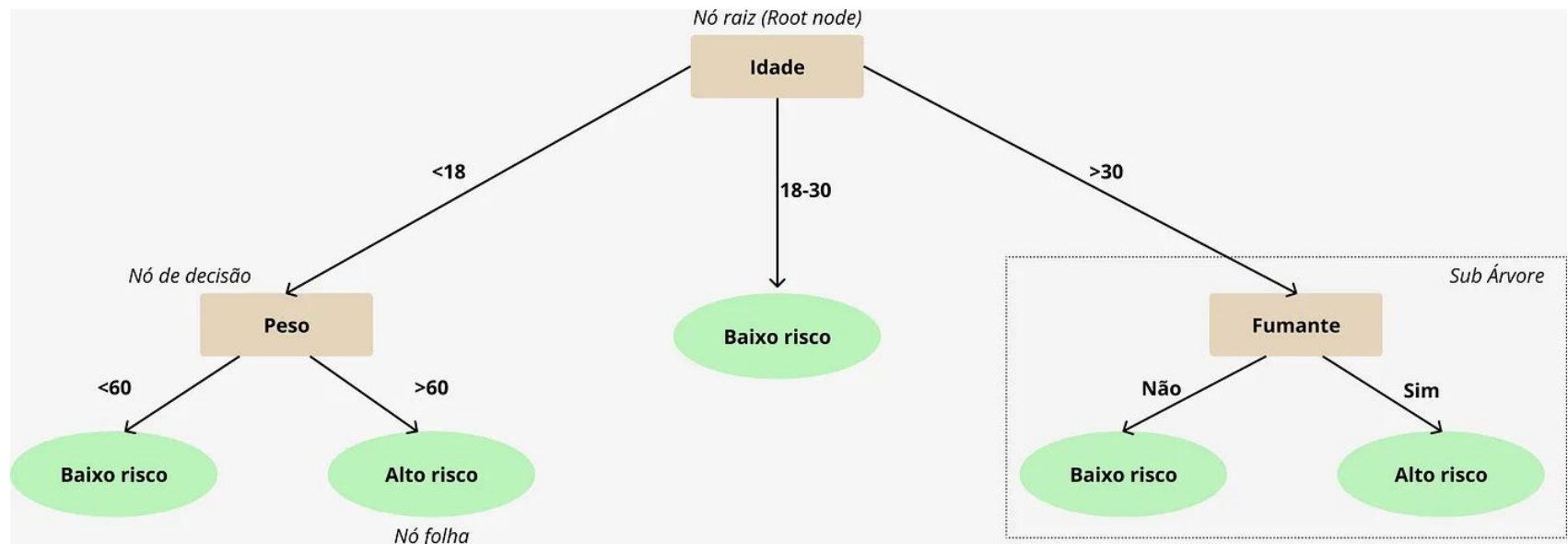
Linear Regression

- Modelo que tenta ajustar uma linha que melhor descreve a relação entre variáveis (X - input e Y-output).
 - Ajusta uma linha reta aos dados para prever um valor numérico baseado em uma ou mais variáveis de entrada.
- Útil para prever valores contínuos (ex: prever preço de casa pelo tamanho).



Árvore de Decisão

- Utiliza um conjunto de regras binárias (0,1) para calcular um Target/Alvo
- Fluxograma em forma de árvore, onde cada nó (não folha) indica um teste feito em um determinado valor
 - A ligação entre os nós representam valores possíveis do teste do nó superior, e as folhas indicam a classe (categoria) a qual o registro pertence.



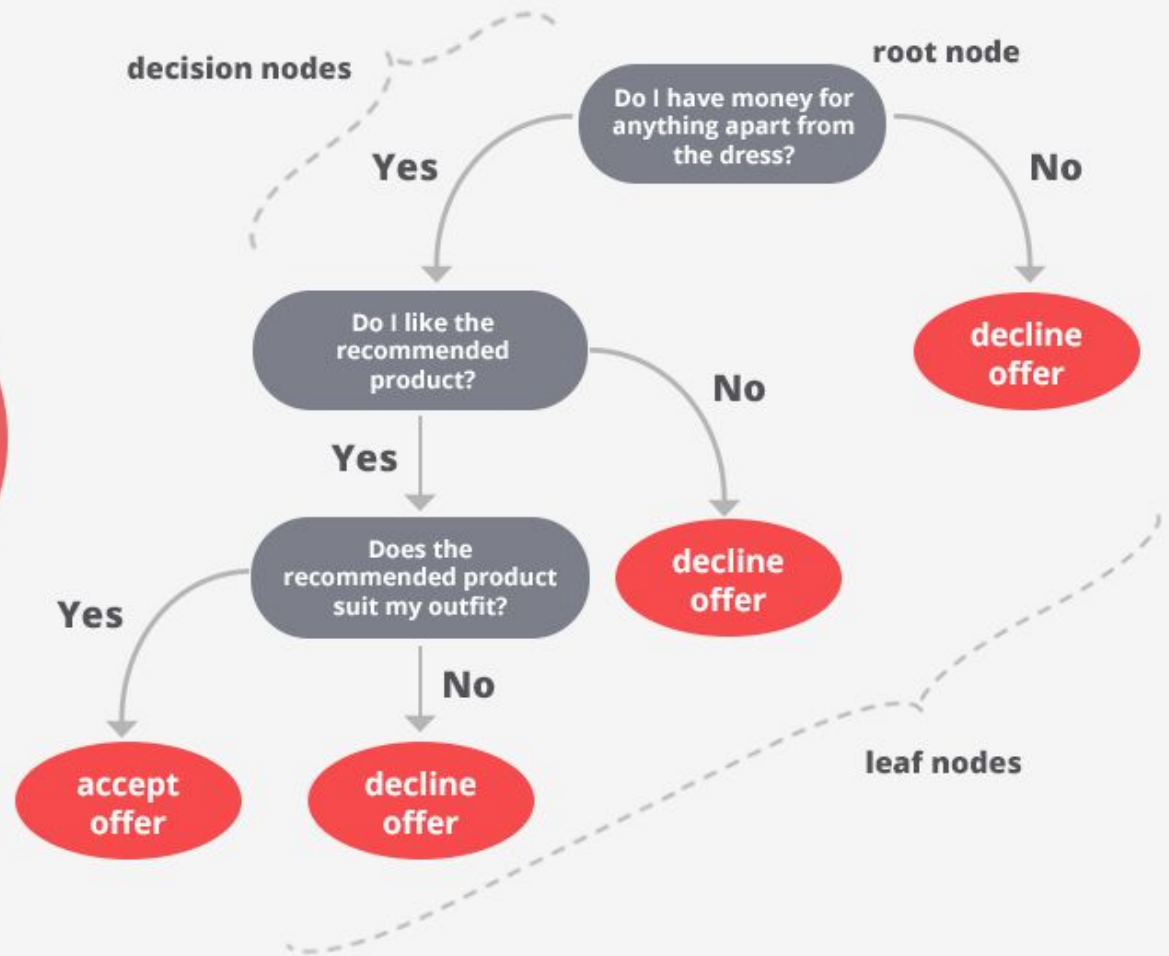
Árvores de Decisão



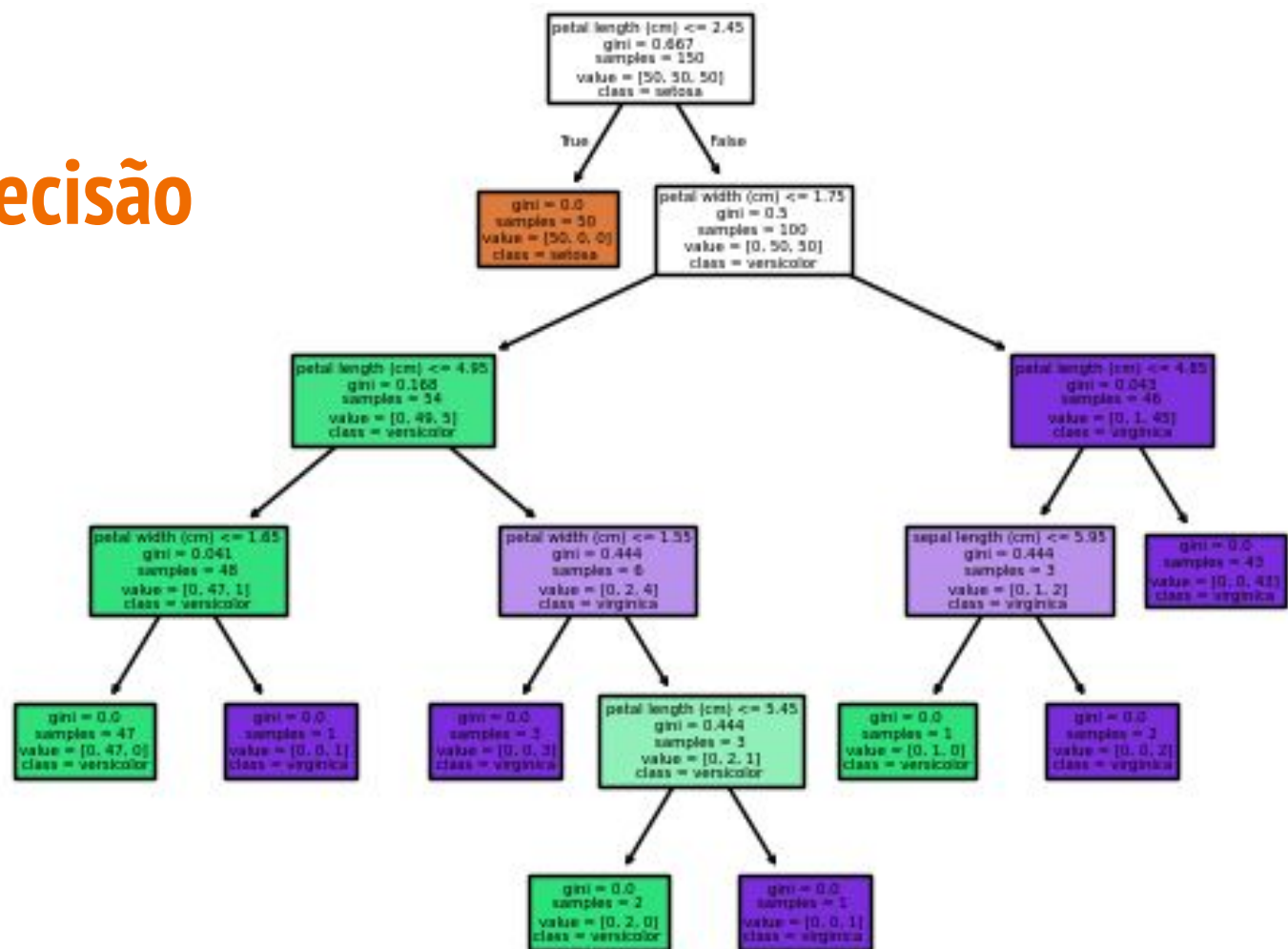
Decision Trees
if rain then...

- Modelo em formato de árvore que faz decisões baseadas em divisões sucessivas dos dados
- Algoritmo que constrói uma árvore onde cada nó é uma decisão baseada em uma característica do dado, separando os exemplos até chegar aos rótulos finais
- Fácil de interpretar e explicar porque o modelo tomou a decisão.

DECISION TREE

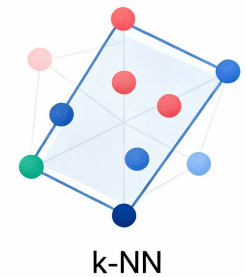


Árvores de Decisão

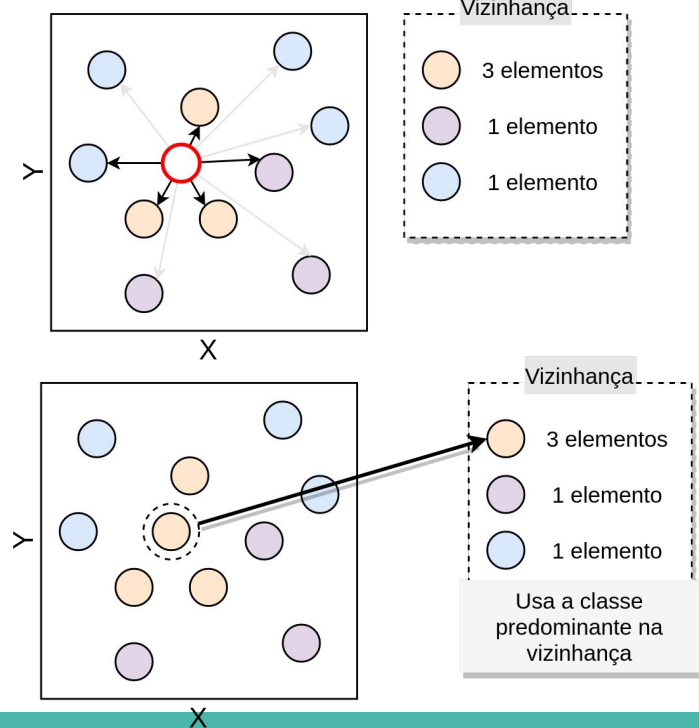
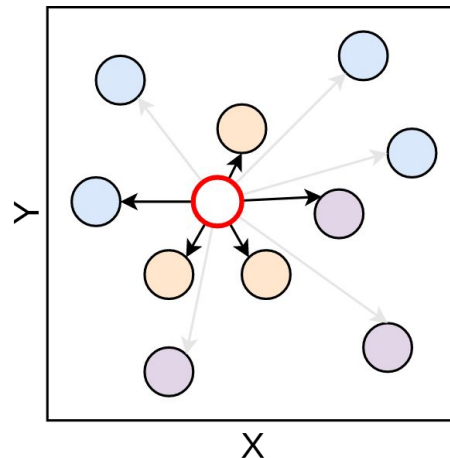
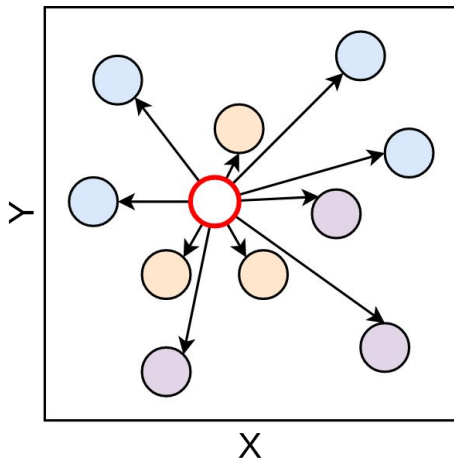


```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
iris = load_iris()
tree_clf = DecisionTreeClassifier(random_state=42)
tree_clf.fit(iris.data, iris.target)
plot_tree(tree_clf, filled=True, feature_names=iris.feature_names, class_names=iris.target_names)
```

k-Nearest Neighbors (k-NN)



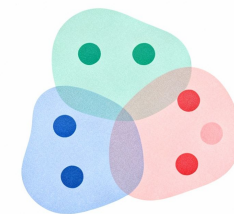
- Classifica um ponto novo pela classe predominante entre seus k vizinhos mais próximos.
- Parâmetro k é configurável, alterando o grau de sensibilidade do modelo
 - controla quão "vizinhos" são considerados



O parâmetro 'k' no k-NN:

- 'k' determina quantos vizinhos próximos são considerados para decidir a categoria do dado
 - $k=3 \rightarrow$ o algoritmo olha as 3 espécies mais próximas para decidir a nova flor (exemplo Iris)
- Esse parâmetro é ajustável e influencia resultado e acurácia.

SVM (Support Vector Machine)



SVM

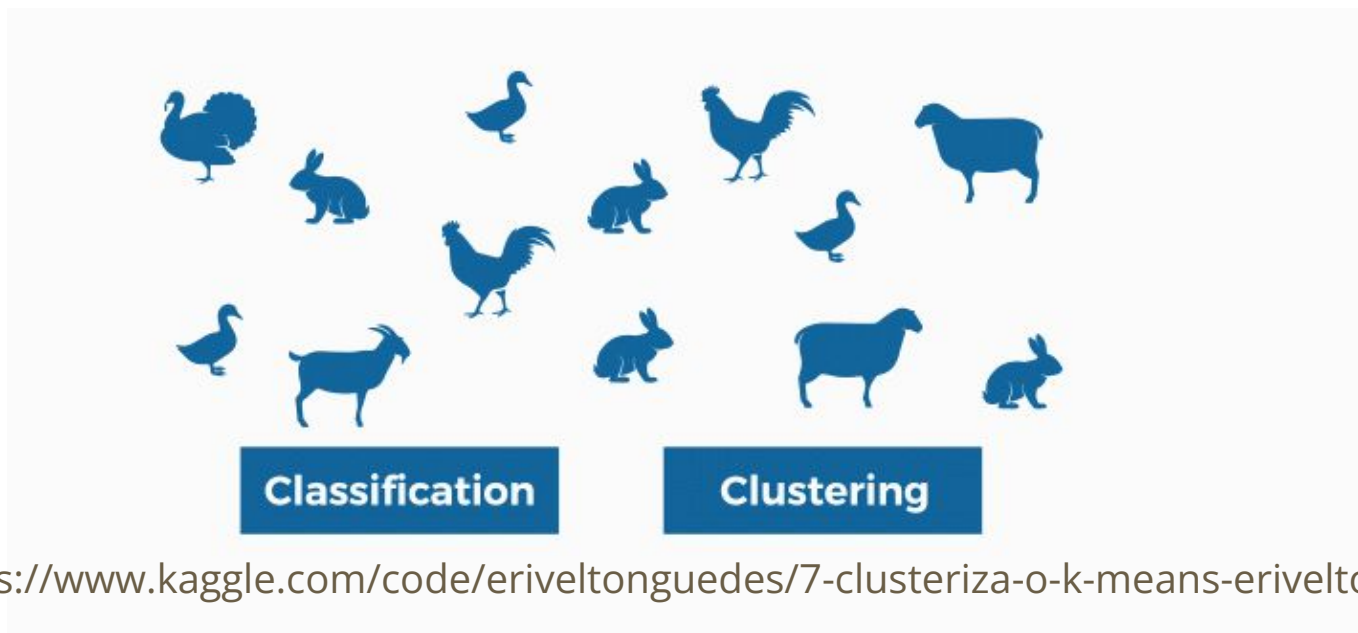
- SVM (Support Vector Machine) é um algoritmo de aprendizado supervisionado para classificação (principalmente binária) e regressão
 - separa dados em classes distintas com alta precisão
- Busca encontrar a melhor fronteira (hiperplano) que separa as classes, maximizando a margem entre os dados classificados.

SVM (Support Vector Machine)

- Vantagens:
 - Fronteiras de decisão flexíveis
 - Menor risco de overfitting (com regularização)
 - Bom desempenho em problemas de texto, imagens e bioinformática
- Limitações:
 - Treinamento lento para conjuntos de dados grandes.
 - Sensível ao ajuste de hiperparâmetros (C, tipo de kernel, gamma).
 - Não gera probabilidades diretamente.
- Aplicações:
 - Reconhecimento de imagens e texto.
 - Bioinformática.
 - Diagnóstico médico.
 - Qualquer problema de classificação com fronteiras não lineares

Aprendizado Não Supervisionado

- Algoritmo recebe dados que **não são rotulados**
- Tem apenas entradas X , sem nenhuma saída y correspondente;
- Não é prever uma resposta correta, mas encontrar padrões, estruturas e relações ocultas nos próprios dados;



Aprendizado Não Supervisionado

- Tarefas mais comuns:
 - Clusterização
 - Agrupar os dados em "*clusters*" ou *conglomerados*, pontos de dados dentro de um mesmo cluster são mais similares entre si do que com os outros *clusters*
 - Agrupar clientes em uma loja em diferentes *segmentos de mercado*; agrupar notícias por *tópico*
 - Redução de Dimensionalidade
 - Simplificar os dados reduzindo o número de variáveis (dimensões), enquanto tenta preservar estrutura e as informações mais importantes

Aprendizado não-supervisionado

🍇 Experiência adquirida: Apenas dados de entrada

Não existe supervisor

Dados de entrada

Fonte: Google images, 2022



- Análise de agrupamentos;
- Estudos de divergência genética em BAG's.

Visualizar e compreender padrões e tendências dentro de um conjunto de dados

Exemplo Supervisionado (k-NN com Iris)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

print("Acurácia:", accuracy_score(y_test, model.predict(X_test)))
```

⇒ Acurácia: 1.0

Aprendizado Não Supervisionado

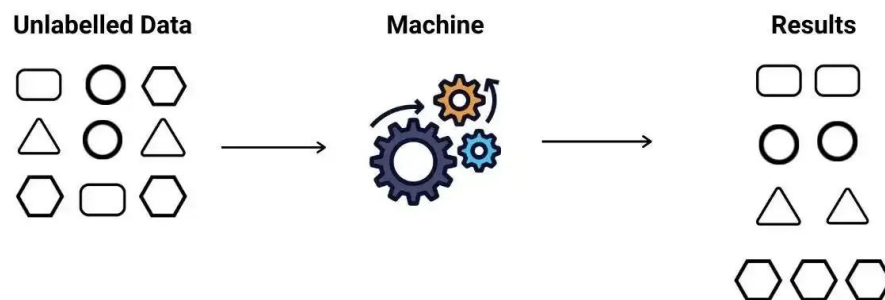
Exemplos:

- Agrupar clientes por perfil de compra
- Reduzir dimensionalidade de imagens (PCA)

Principais algoritmos:

- k-Means
- PCA

Unsupervised Learning



k-Means

- Algoritmo que separa os dados em k grupos (clusters) baseados na distância entre pontos e centróides
- Inicialização de k centroides, atribuição de pontos, atualização dos centróides, repetição até convergência
- Objetivo: minimizar a soma das distâncias entre pontos e seus centroides
- Não usa rótulos, agrupando dados que são similares.

O que é número de clusters em k-Means?

- Clusters são grupos que o algoritmo tenta formar no conjunto de dados.
- O número de clusters define quantos grupos diferentes queremos que ele ache.
- Definir clusters errados pode “juntar coisas diferentes” ou “separar coisas iguais”.

PCA (Análise de Componentes Principais)

- Técnica que reduz a dimensionalidade dos dados mantendo a maior parte da variabilidade.
- Facilita visualização e acelera outros modelos
- Compactar os dados sem perder muito da informação relevante, facilitando a análise e a aplicação de métodos como K-means

Número de clusters no k-Means:

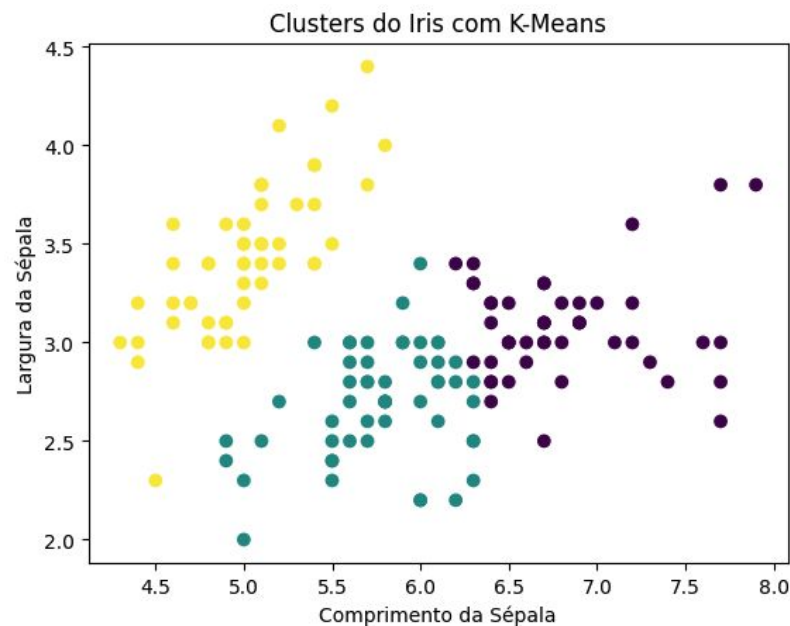
- Número de grupos que o algoritmo vai formar automaticamente no conjunto de dados.
- Ajustar esse número influencia onde os pontos são agrupados, e não existe uma regra única para escolher o ideal.

Exemplo Não Supervisionado (k-Means com Iris)

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

X = iris.data[:, :2] # duas features para visualizar
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)

plt.scatter(X[:,0], X[:,1], c=kmeans.labels_, cmap="viridis")
plt.xlabel("Comprimento da Sépala")
plt.ylabel("Largura da Sépala")
plt.title("Clusters do Iris com K-Means")
plt.show()
```



0 Ciclo de um Projeto de Machine Learning

1. **Coleta de dados:** O que será aprendido? De onde vêm os exemplos para aprender?
2. **Pré-processamento:** Tratar dados faltantes, normalizar.
3. **Divisão treino/teste:** Separar parte dos dados para validar se o modelo “realmente aprendeu”.
4. **Escolha e treinamento de modelos:** Testar diferentes algoritmos (KNN, árvore, SVM, redes neurais).
5. **Avaliação:** Como medir o sucesso? (acurácia, RMSE, etc).
6. **Uso em produção:** Quando o modelo aprende durante o uso? Como monitorar?

O CICLO DE UM PROJETO DE MACHINE LEARNING



Por que Importa Confiar ou Duvidar de uma Previsão?

- Em aplicações críticas (saúde, indústria) um erro pode causar grandes problemas.
 - É importante conhecer as limitações dos modelos e sempre validar os resultados.
- Aprendizado de máquina ajuda, mas não substitui o raciocínio humano.

Diferenças

- **Supervisionado** (ex.: Árvore de Decisão, k-NN):
 - O objetivo é prever algo conhecido, usando exemplos rotulados.
 - Por exemplo: prever a espécie da flor sabendo as medidas das pétalas e sépalas. Você treina o algoritmo com as respostas corretas (labels), e depois avalia se ele acerta na predição de novas amostras.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
iris = load_iris()
tree_clf = DecisionTreeClassifier(random_state=42)
tree_clf.fit(iris.data, iris.target)
pred = tree_clf.predict([iris.data[0]])
print("Exemplo supervisionado: previsão da espécie:", iris.target_names[pred][0])
```

Exemplo supervisionado: previsão da espécie: setosa

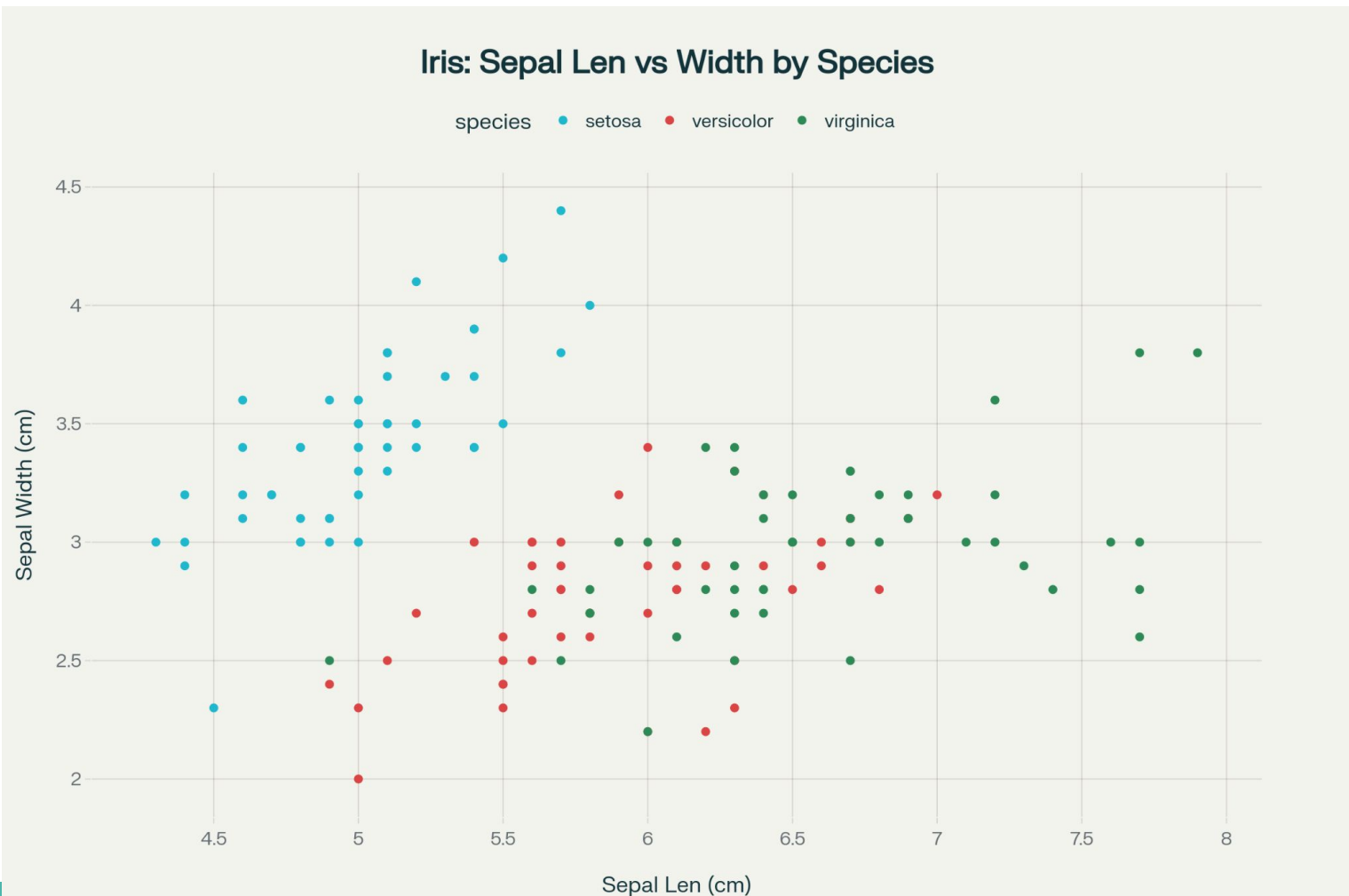
Diferenças

- **Não supervisionado** (ex.: k-Means, PCA)
 - O objetivo é encontrar padrões ou agrupar os dados SEM saber a resposta correta.
 - k-Means agrupa amostras similares, podendo sugerir – por exemplo – que há três grupos principais.
 - PCA reduz as dimensões, permitindo visualizar/quais grupos se formam naturalmente

```
# k-Means sem usar labels reais
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(iris.data)
print("Exemplo não supervisionado: clusters encontrados:", set(labels))
```

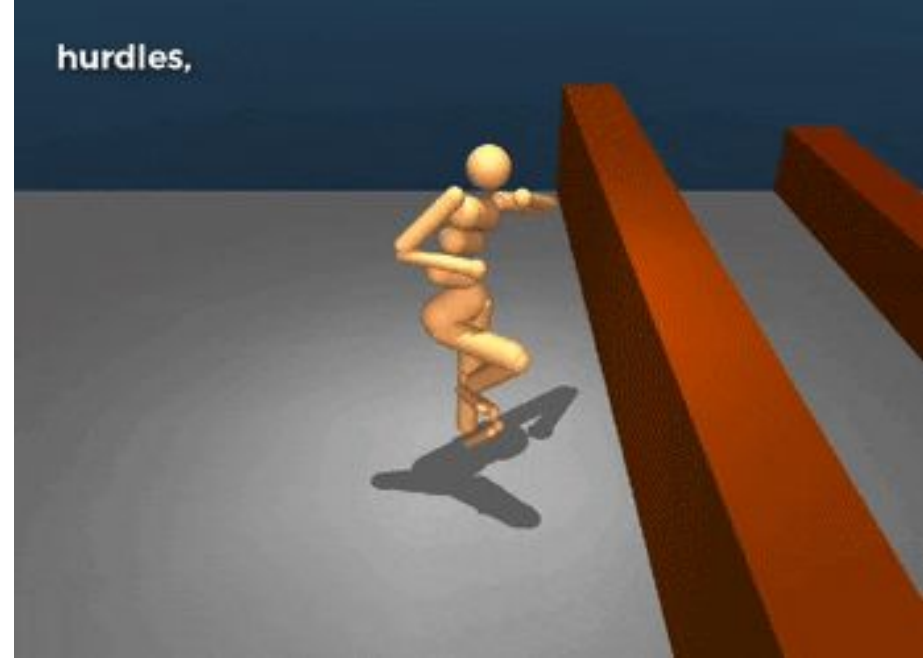
```
Exemplo não supervisionado: clusters encontrados: {np.int32(0), np.int32(1), np.int32(2)}
```

Tipo de Algoritmo	O que faz	Exemplo prático
Supervisionado	Aprende com exemplos rotulados	Prever espécie da flor
Não supervisionado	Encontra grupos ou padrões sem saber as respostas certas	Agrupar flores por medidas



Aprendizado por Reforço

- Baseado em feedbacks de interações anteriores.
- Esse tipo de aprendizagem melhora continuamente seu modelo
- Utiliza o conceito de recompensa e punição
 - É dada uma recompensa se houver acerto e uma punição se houver erro
- utilizado nos modelos que aprendem de jogos (Gamão, Xadrez, Go) e navegação de robôs



Fonte:

<https://towardsdatascience.com/reinforcement-learning-markov-decision-process-part-2-96837c936ec3>

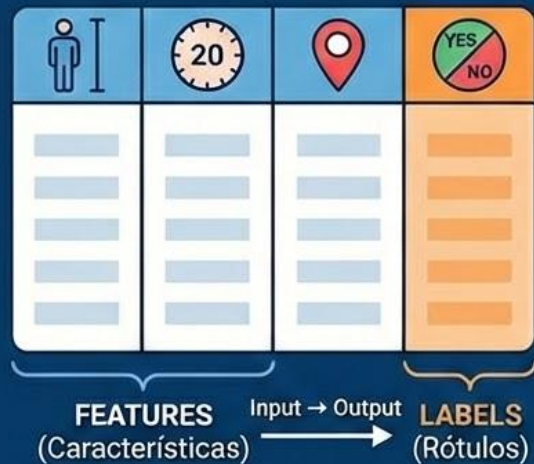
Aprendizado por Reforço

- Componentes chave:
 - Agente
 - Ambiente
 - Estado
 - Ação
 - Recompensa
- *Exemplo:* Treinar uma IA para jogar xadrez, ensinar robô a andar; otimizar o sistema de refrigeração de um data center

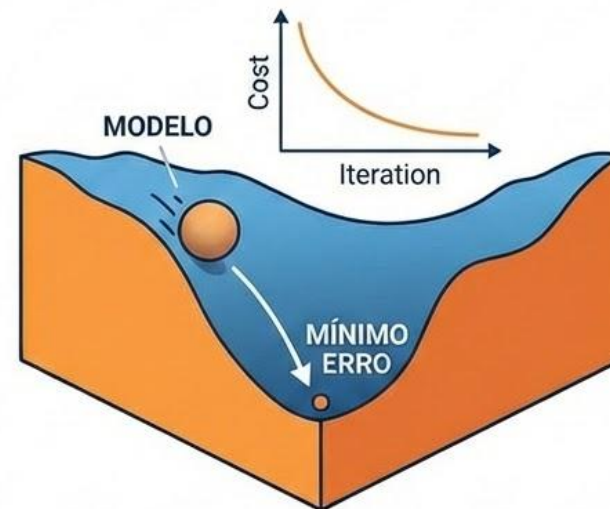
Conceitos Fundamentais do Aprendizado de Máquina

- **Anatomia dos Dados:** Features e Labels
- **Objetivo do Treinamento:** a função de custo
- **Grande desafio:** overfitting e underfitting
- Separação de dos dados

ANATOMIA DOS DADOS: FEATURES E LABELS



OBJETIVO DO TREINAMENTO: A FUNÇÃO DE CUSTO



GRANDE DESAFIO: OVERFITTING E UNDERFITTING



SEPARAÇÃO DE DADOS



Anatomia dos Dados: Features e Labels

- Todo problema de aprendizado supervisionado começa com dados
- Grande tabela, onde as colunas representam as variáveis e as linhas representam cada observação ou exemplo
 - **Features** (características ou atributos)
 - São as variáveis de entrada que o modelo usará para fazer uma previsão.
 - É o X da nossa função $y = f(X)$
 - São as "pistas" que o modelo usa
 - **Label** (rótulo ou alvo)
 - Variável de saída que estamos tentando prever
 - É o y , "resposta correta" que o modelo tenta aprender

OBSERVAÇÃO / EXEMPLO

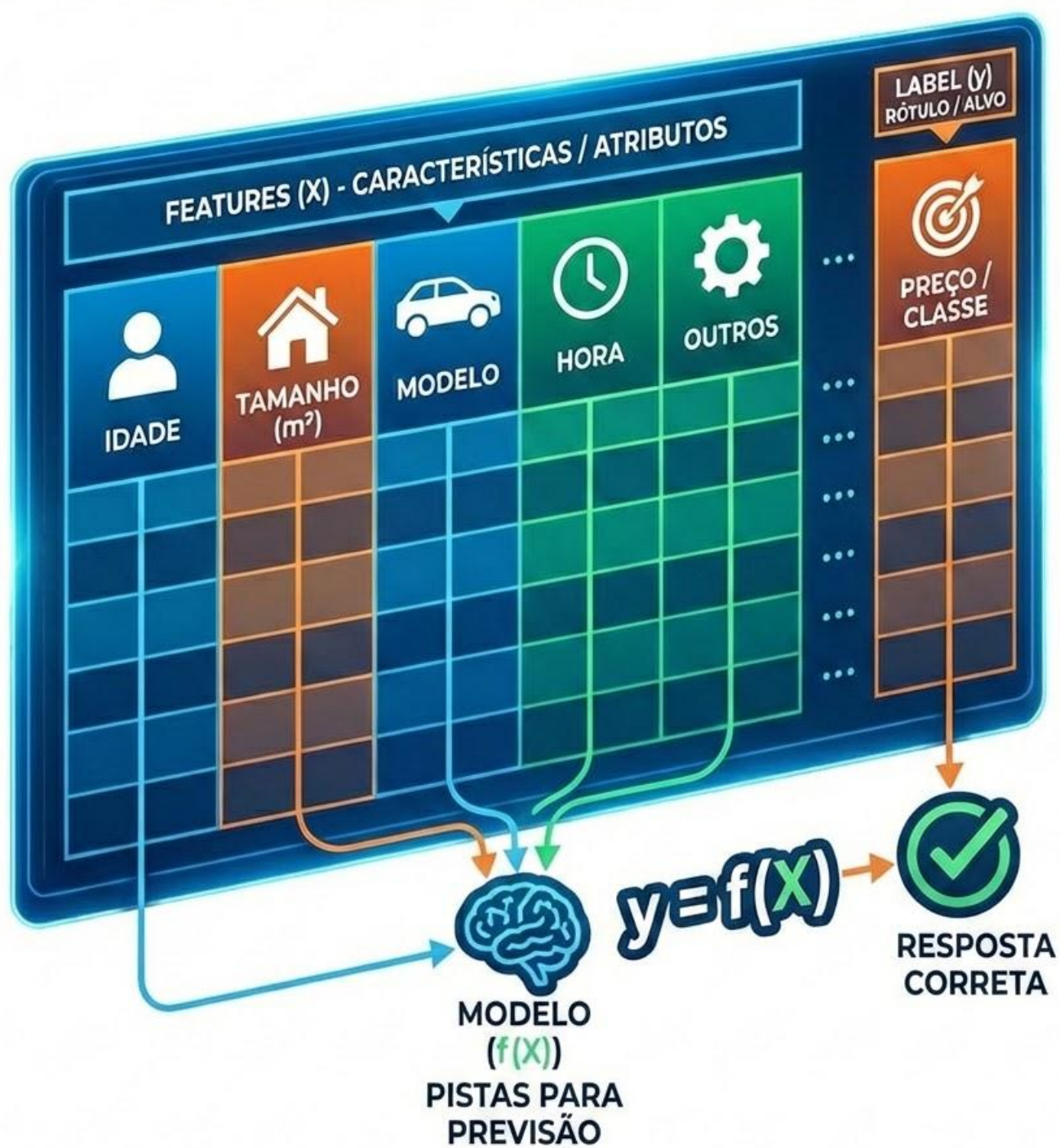


Tabela prever preço de uma casa

Área (m ²)	Nº. Quartos	Dist. Centro (km)	Preço (R\$)
120	3	5	500.000
80	2	10	320.000
200	4	2	850.000

Objetivo do Treinamento: a função de custo

- Define o "erro" do modelo entre previsão e valor real
- O treinamento busca minimizar essa função

- Exemplo: Erro Quadrático Médio (MSE)

- Fórmula:

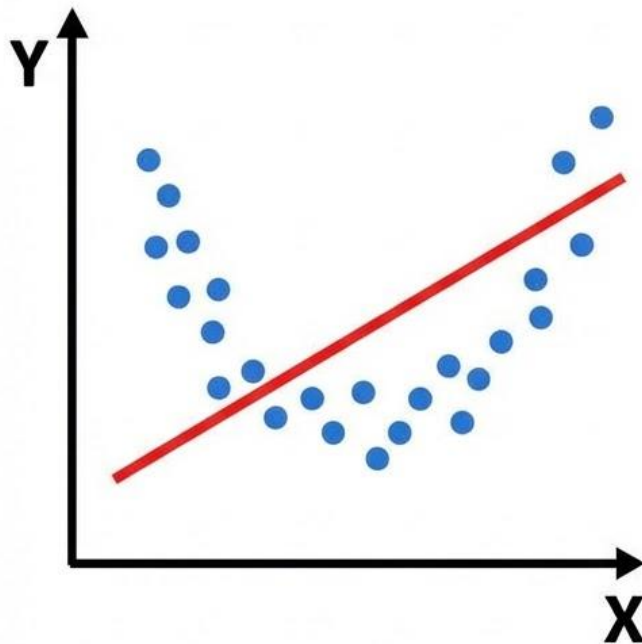
$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Analogia do jogo "quente ou frio" para ajustar os parâmetros

Overfitting e Underfitting

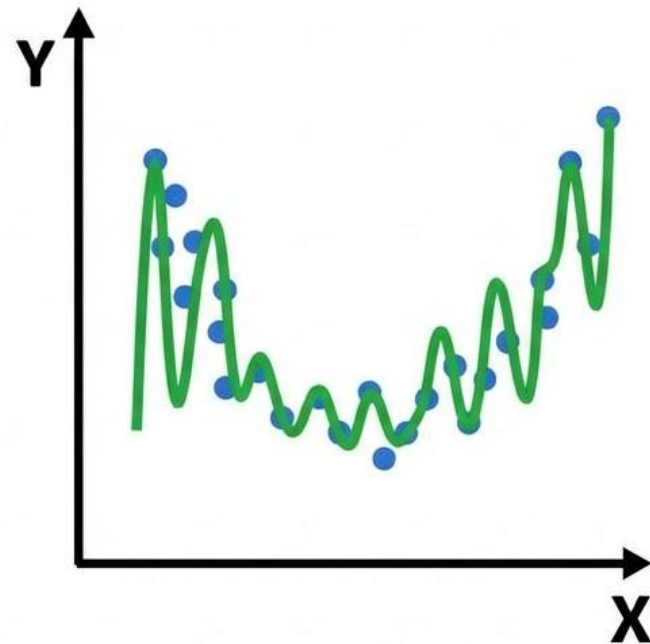
- **Underfitting:** modelo simples demais, não captura a complexidade dos dados, aprende pouco
- **Overfitting:** modelo complexo demais, decora padrões e ruídos, falha em dados novos
- **Objetivo:** Encontrar o modelo que generalize bem (ponto ótimo)

UNDERFITTING (Alto Viés)



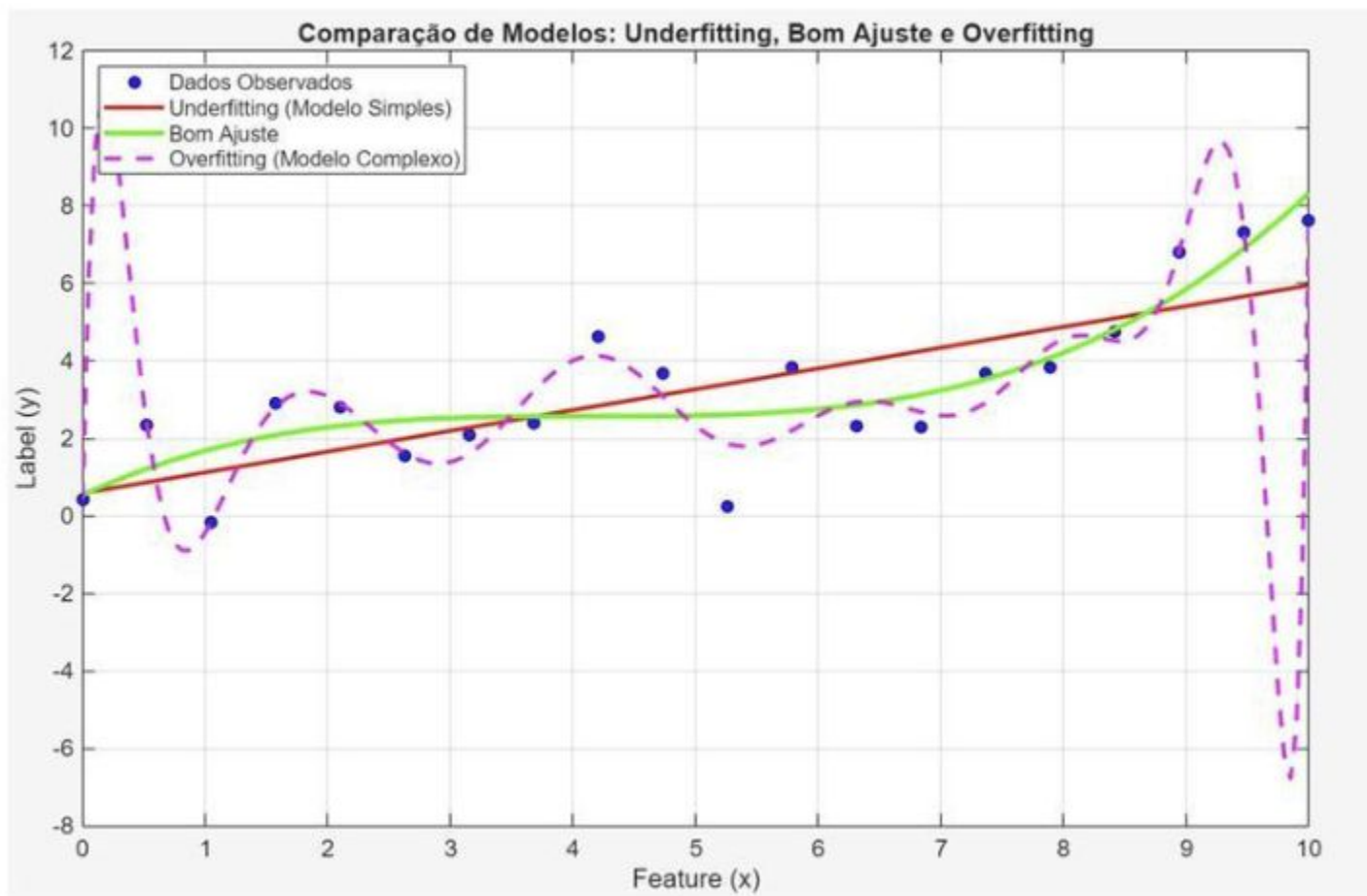
Modelo **simples demais**. Não aprende os padrões.

OVERFITTING (Alta Variância)



Modelo **complexo demais**. Decora os dados, não generaliza.

Ilustração de underfitting, bom ajuste e overfitting



Separação dos Dados (Importância)

- Testar o modelo com dados nunca vistos para detectar overfitting
- Evitar usar todos dados no treino para garantir avaliação imparcial

Conjuntos de Dados

1. **Treino** (Training Set): 70-80%, para aprender os padrões
2. **Validação** (Validation Set): 10-15%, para ajustar hiperparâmetros durante desenvolvimento
3. **Teste** (Test Set): 10-15%, usada apenas uma vez para avaliação final e imparcial

O que é um dataset?

- Conjunto organizado de dados, como uma tabela.
- Cada linha é um exemplo; cada coluna, uma característica do exemplo.
- Pode ser um dataset didático (ex: Iris) ou um conjunto real, coletado para análise (ex: dados médicos, industriais).

Dataset Real

- Dados coletados de situações da vida real, muitas vezes maiores e mais complexos.
- Usar datasets reais ajuda a aprender a trabalhar com dados variáveis e com possíveis erros.

Exemplos: dados de sensores, informações de clientes, registros médicos

Sites para Datasets Gratuitos

- **Kaggle Datasets**

- Maior repositório online, milhares de bases para classificação, regressão e agrupamento.
- <https://www.kaggle.com/datasets>

- **UCI Machine Learning Repository**

- Clássico em IA, conjuntos para tarefas supervisionadas e não supervisionadas, fácil importação.
- <https://archive.ics.uci.edu>

- **OpenML**

- Plataforma colaborativa, busca avançada por tipo de problema, permite compartilhar resultados.
- <https://www.openml.org>

Outras opções úteis

- **Sigma Open Datasets**

- Grande coleção de dados abertos para IA, fácil download.
- <https://sigma.ai/open-datasets/>

- **Google Dataset Search**

- Busca datasets de diversas fontes acadêmicas, governamentais e web.
- <https://datasetsearch.research.google.com/>

- **365DataScience**

- Listagens dos melhores datasets, prontos para projetos e aprendizagem.
- <https://365datascience.com/trending/public-datasets-machine-learning/>

O que é um algoritmo e parâmetro?

- Algoritmo é o conjunto de passos para resolver um problema (ex: k-NN e k-Means).
- Parâmetros são configurações que ajustam o algoritmo
 - Ex.: número de vizinhos 'k' em k-NN, número de clusters em k-Means

Aprendizado Supervisionado - Parâmetros Algoritmos

- **Regressão Logística**

- `C`: controla força da regularização
- `max_iter`: número máximo de iterações
- `solver`: algoritmo de otimização

- **Random Forest**

- `n_estimators`: número de árvores
- `max_depth`: profundidade máxima das árvores
- `min_samples_split`: mínimo de amostras para dividir nó

- **Support Vector Machine (SVM)**

- `kernel`: tipo de função núcleo (linear, rbf, etc)
- `C`: penalização de erro
- `gamma`: influência do alcance da função kernel

Aprendizado Não Supervisionado - Parâmetros Algoritmos

- **K-means**

- `n_clusters`: número de grupos
- `init`: método de inicialização (k-means++, random)
- `max_iter`: máximo de iterações

- **PCA (Análise de Componentes Principais)**

- `n_components`: número de componentes a manter
- `svd_solver`: algoritmo para decomposição

- **DBSCAN**

- `eps`: raio máximo de vizinhança para agrupamento
- `min_samples`: mínimo de pontos para formar cluster

Visualização e Avaliação dos Resultados

- Depois que o algoritmo roda, avaliamos os resultados para ver se ele acertou ou agrupou bem.
- Visualizações (gráficos ou tabelas) ajudam a entender onde o modelo funcionou ou falhou.

Supervisionado (classificação/regressão)

- Visualização:
 - Gráficos de barras com resultados por classe
 - Matriz de confusão (acertos x erros por classe)
 - Curva ROC e AUC (binário/multiclasse)
 - Dispersão predição vs. real (regressão)
- Avaliação:
 - Acurácia, precisão, recall e F1-score (classificação)
 - Erro médio absoluto (MAE), erro quadrático médio (MSE), R^2 (regressão)
 - Matriz de confusão para analisar tipos de erro

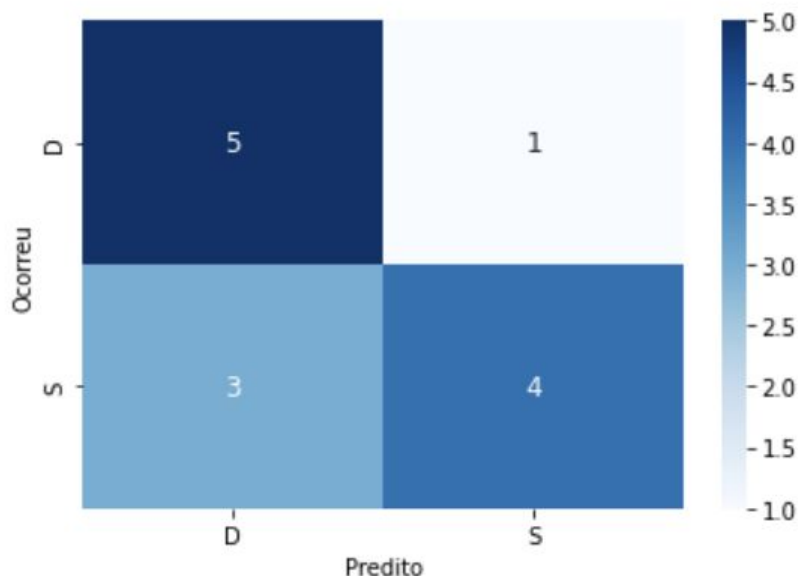
Matriz de Confusão: Pontos Essenciais

- Ferramenta que mostra comparativo entre as classes reais e as predições feitas pelo modelo de classificação.
- Ajuda a identificar acertos e erros em cada classe de modo detalhado.

Por que usar?

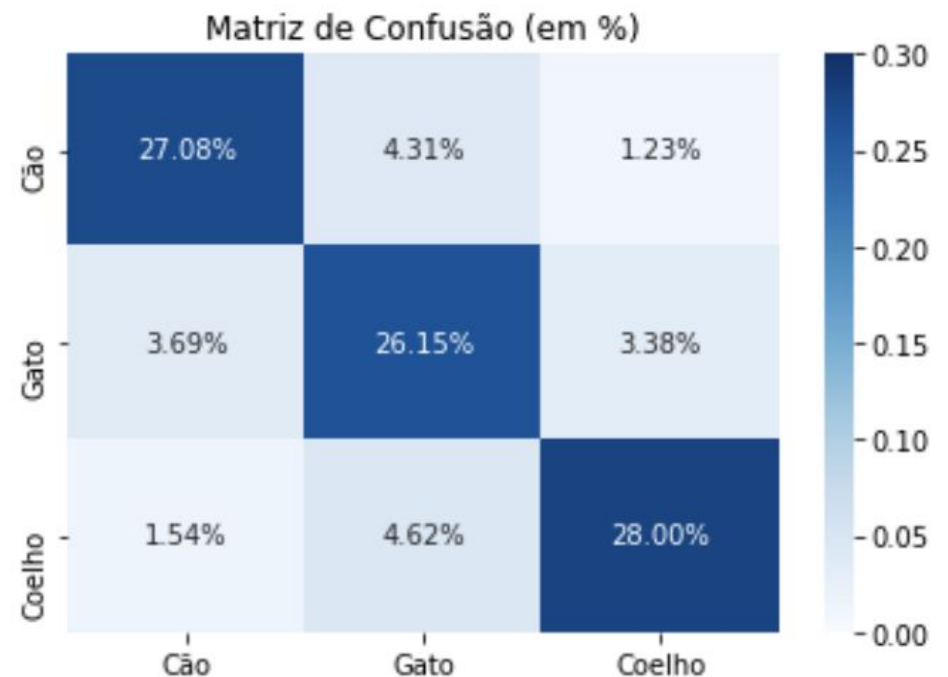
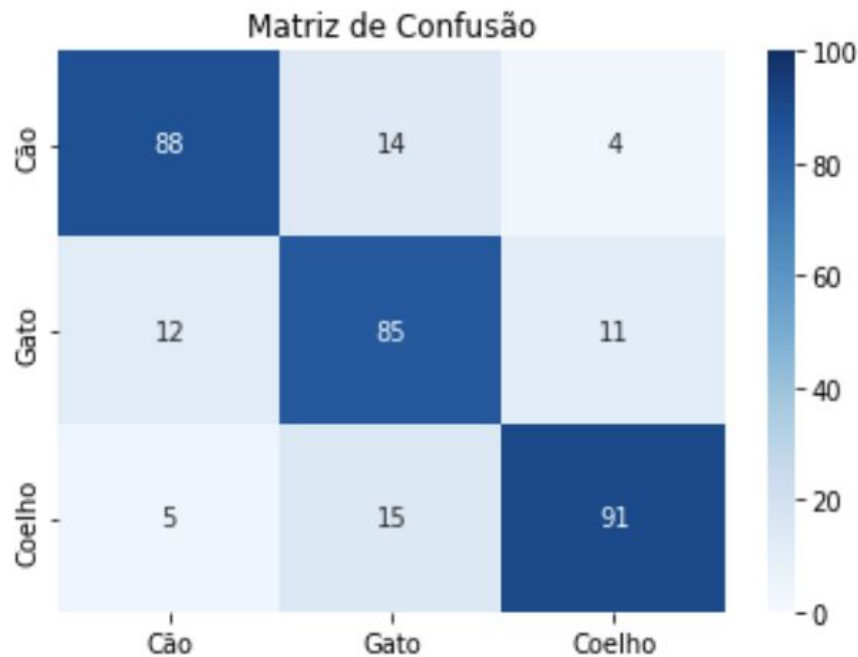
- Permite saber não só a acurácia total, mas o tipo de erro cometido em cada situação.
- Essencial em contextos onde consequências dos erros são diferentes (ex: saúde, fraude).

Matriz de Confusão: Pontos Essenciais



- Categorias:
 - Verdadeiro Positivo (VP): acerto ao prever ocorrência.
 - Falso Positivo (FP): erro ao prever ocorrência que não aconteceu (tipo 1).
 - Falso Negativo (FN): erro ao não prever ocorrência que aconteceu (tipo 2, normalmente mais grave).
 - Verdadeiro Negativo (VN): acerto ao prever que não ocorreu.

Matriz de Confusão: Pontos Essenciais



Matriz de Confusão

- Tabela usada para avaliar o desempenho de modelos de classificação
- Mostra a quantidade de acertos e erros do modelo ao comparar as previsões com as classes reais dos dados
 - Cada **linha** representa uma classe **real**.
 - Cada **coluna** representa uma classe **prevista** pelo modelo.
 - **Acertos** aparecem na **diagonal** principal da matriz

	Pred A	Pred B	Pred C
True A	Correct	Misclassified	Misclassified
True B	Misclassified		Misclassified
True C	Misclassified	Misclassified	

	Previsto A	Previsto B	Previsto C
Real A	5	1	0
Real B	0	7	2
Real C	1	0	6

Avaliação - Acurácia

- Mede quantas vezes o modelo acertou a classe dos exemplos comparado ao total de previsões feitas;
- $\text{Acurácia} = (\text{Número de acertos}) \div (\text{Total de exemplos avaliados})$

Exemplo:

- Se o modelo fez 100 previsões, acertando 90 e errando 10, a acurácia é:
- $\text{Acurácia} = 90 / 100 = 0,90$ ou 90%

Avaliação - Precisão

- Mede quantas das previsões positivas feitas pelo modelo realmente são positivas.

Precisão = (verdadeiros positivos) ÷ (total previsto como positivo)

- Importante quando erros positivos trazem alto custo (ex: diagnóstico de doenças).

Avaliação - Recall

- Mede quantas das amostras positivas reais foram corretamente identificadas pelo modelo.

$\text{Recall} = (\text{verdadeiros positivos}) \div (\text{total de positivos reais})$

- Fundamental quando é importante capturar todos os casos positivos (ex: triagem de fraudes).

Avaliação - F1-score

- Média harmônica entre precisão e recall
- Resume o equilíbrio entre precisão e recall, útil em conjuntos desbalanceados.

$$(F1 = 2 \cdot \frac{\text{precisão} \cdot \text{recall}}{\text{precisão} + \text{recall}})$$

Prática em Python

- Função `train_test_split` da biblioteca `scikit-learn` para separar dados
 - Principais argumentos da função:
 - `test_size`: proporção dos dados reservada para teste (ex: 0.2 para 20%).
 - `train_size`: proporção dos dados para treino (opcional, normalmente o restante).
 - `random_state`: semente para garantir reprodutibilidade.
 - `shuffle`: embaralha os dados antes de dividir (padrão: `True`).

Exemplo prático: separando em treino, validação e teste

```
from sklearn.model_selection import train_test_split
import numpy as np
```

```
# Exemplo: dataset fictício com 100 amostras
```

```
X = np.arange(100).reshape((100, 1))
```

```
y = np.arange(100)
```

```
# 1ª divisão: separa 20% para teste, 80% para treino+validação
```

```
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# 2ª divisão: do restante, separa 25% para validação (25% de 80% = 20% do total)
```

```
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25,
random_state=42)
```

```
print(f"Treino: {X_train.shape[0]} amostras")
```

```
print(f"Validação: {X_val.shape[0]} amostras")
```

```
print(f"Teste: {X_test.shape[0]} amostras")
```

Saída esperada:

- **Treino: 60 amostras**
- **Validação: 20 amostras**
- **Teste: 20 amostras**

Exercício Prático

- Aplicar divisão de dados usando `train_test_split` com dataset simples (ex: Boston Housing, Iris)
- Avaliar o desempenho de um modelo simples (ex: regressão linear) nos conjuntos

Referências

- Russell & Norvig. Inteligência Artificial (abordagem moderna e fundamentos teóricos).
- Mitchell, T. Machine Learning (definição, triade T-E-P original).
- Géron, A. Hands-On Machine Learning (exemplos reais com scikit-learn).
- Müller & Guido. Introduction to Machine Learning with Python (aplicações em Python/Colab).
- Raschka. Python Machine Learning (abordagem aplicada).
- Chollet. Deep Learning with Python (para a próxima unidade de redes neurais).
- Halal, A. Inteligência Artificial: do zero ao especialista